

Electravelin

Documentacion tecnica del sistema

Planificador de viajes para vehiculos electricos en Chile

Version: MVP v2

Estado: implementacion local validada

Fecha de emision: 3 de octubre de 2026

1. Proposito y alcance

Electravelin permite estimar un viaje en vehiculo electrico entre ciudades chilenas, calcular una ruta vial, proyectar el consumo de bateria y sugerir paradas de carga compatibles. El MVP v2 opera localmente con servicios de mapa y un catalogo propio de vehiculos y electrolinerias.

El sistema esta orientado a planificacion informativa: la disponibilidad, los precios y la potencia de las estaciones deben confirmarse por el conductor antes de viajar.

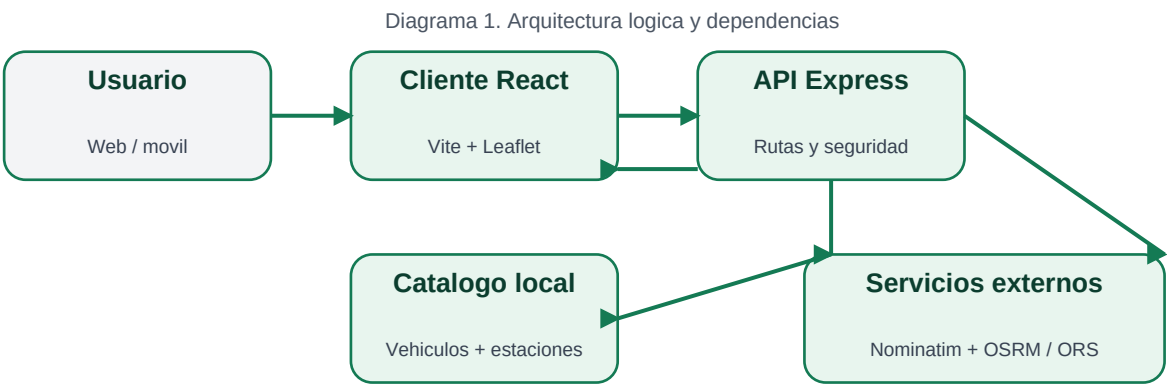
Capacidades entregadas

Capacidad	Implementacion
Planificacion geografica	POST /api/routes/plan con origen, destino, vehiculo y estado de carga (SoC).
Mapa	Leaflet dibuja polilinea, origen, destino, paradas y ajuste automatico de zoom.
Bateria y costos	Reserva minima de 15%, energia, tiempo de carga, costo estimado y SoC de llegada.
Carga inicial	Con SoC bajo se propone una electrolinera compatible cercana al origen antes de iniciar el viaje.
Catalogo	Vehiculos, electrolinerias, filtros, favoritos y administracion autenticada.
Operacion	Health checks, request IDs, logs estructurados y configuracion local.

2. Arquitectura

La aplicacion se compone de un cliente React/Vite y una API Express. El cliente nunca recibe claves de proveedores: el backend realiza la geocodificacion y el ruteo, calcula el plan de carga y devuelve un contrato JSON listo para visualizacion.

Vista de componentes



Flujo principal

Usuario -> React/Vite (5173) -> Express API (3001) -> geocodificador / proveedor de ruteo -> motor de bateria + catalogo de estaciones -> respuesta JSON -> mapa y resultados.

Modulo	Responsabilidad
client/src	Formulario de viaje, resultados, mapa Leaflet, catalogo, perfil y panel administrador.
server/src/index.js	Inicializacion de Express, CORS, rutas, logging y health checks.
services/routingProviders.js	Adaptadores de geocodificacion y ruteo. OSRM es el fallback local sin clave.
services/routePlanningService.js	Seleccion de estaciones, reserva de bateria, paradas, costos y advertencias.

Modulo	Responsabilidad
services/batteryService.js	Calculos de consumo, compatibilidad de conectores y tiempos de recarga.
data/*.json	Catalogos locales de vehiculos, estaciones y ciudades para desarrollo.
supabase/	Esquema y recursos para persistencia/autenticacion cuando se habilite el entorno gestionado.

3. Modelo UML y entidad-relacion

Los diagramas siguientes representan las responsabilidades principales del backend y el esquema de persistencia previsto en la migracion de Supabase. La implementacion local usa un store JSON con las mismas entidades funcionales.

Diagrama UML de servicios

Diagrama 3. UML simplificado de los servicios principales

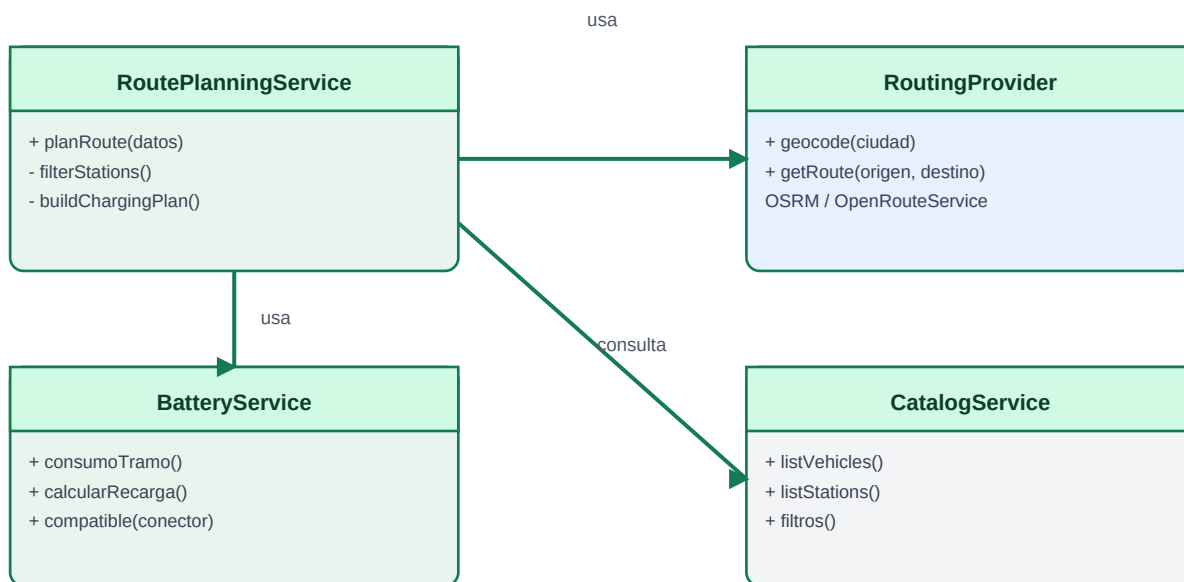
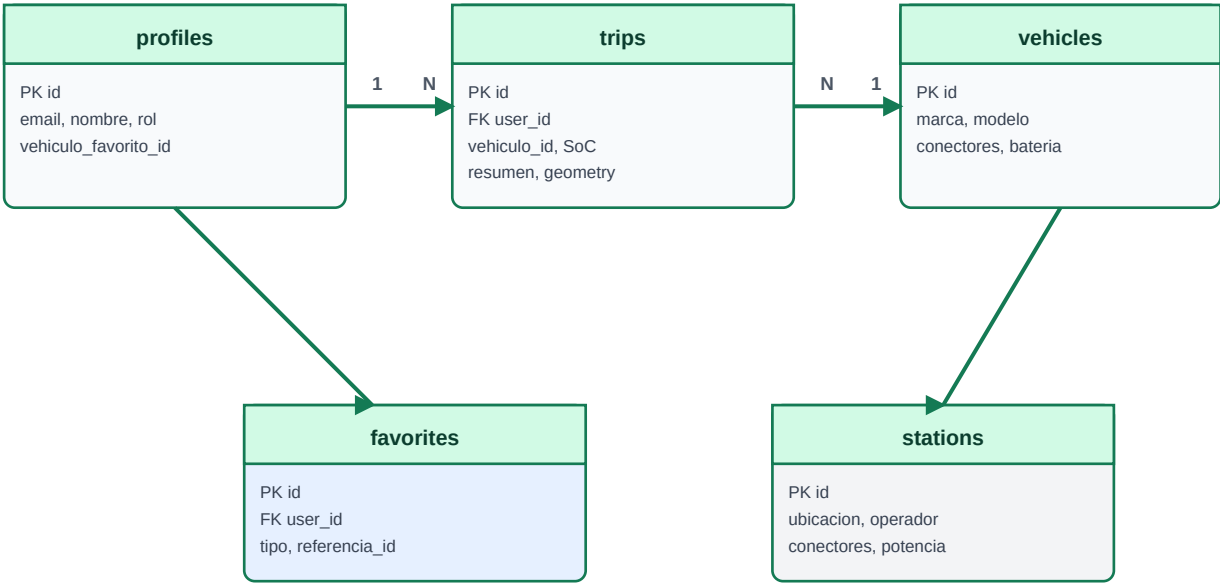


Diagrama entidad-relacion

Diagrama 4. Modelo entidad-relacion (persistencia Supabase)



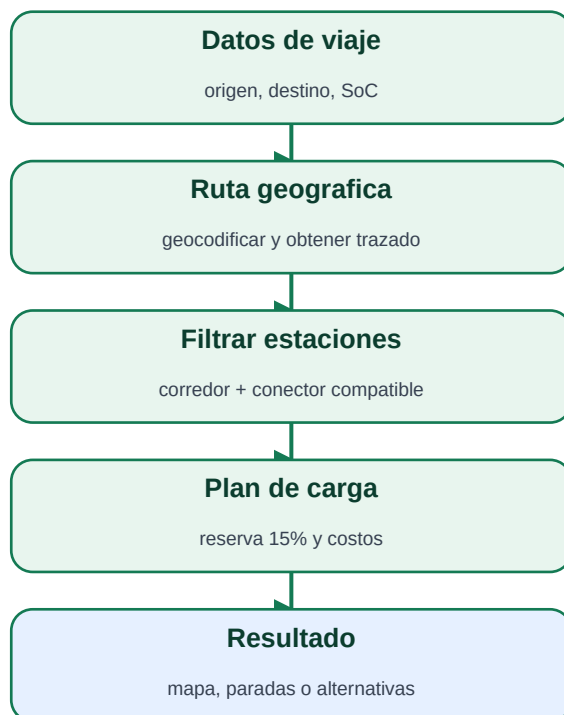
favorites usa referencia_id polimorfica para vehiculo o estacion; audit_log registra acciones administrativas.

4. Motor de ruteo y carga

El servicio obtiene la geometría vial, proyecta las estaciones sobre el trazado y conserva solo las que son compatibles con el conector del vehículo, están disponibles y se encuentran dentro de un corredor de 35 km. Luego avanza por tramos manteniendo una reserva mínima de 15% de SoC.

Flujo operativo

Diagrama 2. Flujo de planificación de un viaje



Reglas relevantes

Regla	Comportamiento
Compatibilidad	Solo se usan estaciones que comparten CCS2 o CHAdEMO con el vehículo.
Reserva	Nunca se planifica un tramo que deje el SoC por debajo de 15%.
Carga objetivo	La parada carga al menos a 80% o al SoC necesario para alcanzar la siguiente parada/destino.
SoC bajo al salir	Se revisan estaciones compatibles a un radio de 8 km del origen y se inserta una parada de carga inicial.
Cobertura insuficiente	Se devuelve respuesta controlada, con advertencia y alternativas en el origen cuando existen.

Ejemplo validado: Santiago -> Talca, Nissan Leaf e+ (CHAdEMO), SoC 20%. El plan incorpora Copec Voltex Santiago Centro como carga de salida; con 80% se propone Shell Recharge Curico como parada intermedia.

5. API y contratos

Endpoint de planificación

POST /api/routes/plan

Cuerpo de solicitud:

```
{"origin":"Santiago","destination":"Talca","vehiculoId":"nissan_leaf_eplus","socInicial":20}
```

La respuesta exitosa contiene **geometry** (LineString), **resumen**, **paradas**, **advertencias**, origen, destino, vehiculo y **estacionesOrigenCompatibles**. Las rutas no factibles se responden como HTTP 200 con **exito:false** y un codigo de dominio, para que la interfaz no se rompa.

Endpoint	Uso
GET /api/health	Disponibilidad basica de la API.
GET /api/ready	Estado de preparacion y proveedor de ruteo configurado.
GET /api/vehiculos	Catalogo publico de vehiculos.
GET /api/estaciones	Catalogo de electrolinerias con filtros.
POST /api/auth/register y /login	Registro e inicio de sesion JWT.
GET/POST /api/viajes	Historial del usuario autenticado.
/api/admin/*	CRUD y auditoria para administradores.

6. Datos, seguridad y observabilidad

Las claves privadas se mantienen exclusivamente en variables de entorno del backend. El proveedor configurado se selecciona mediante ROUTING_PROVIDER; si no existe ORS_API_KEY, el sistema utiliza OSRM como fallback. Las respuestas de readiness informan configuracion sin exponer secretos.

Variables de entorno relevantes

Variable	Uso
PORT	Puerto del servidor (3001 por defecto).
JWT_SECRET	Firma de sesiones. Debe definirse fuera de desarrollo.
ROUTING_PROVIDER	osrm u openrouteservice.
OSRM_BASE_URL	URL del servicio OSRM, si se requiere reemplazar la publica.
ORS_API_KEY	Clave privada de OpenRouteService cuando se utiliza ese proveedor.
VITE_API_URL	URL de API consumida por el cliente, si se despliega separado.

Cada respuesta HTTP recibe un x-request-id. El logger registra eventos estructurados y redacta campos sensibles. Los errores de proveedores se traducen a mensajes seguros para la UI sin revelar claves ni detalles internos.

7. Manual de usuario

Planificar un viaje

Paso	Accion del usuario	Resultado esperado
1	En "Planifica tu viaje", selecciona ciudad de origen y destino.	El formulario valida que ambas ciudades sean distintas.
2	Elige un vehiculo electrico y ajusta la bateria inicial.	Se muestran capacidad, consumo y conectores compatibles.
3	Presiona "Calcular ruta".	El sistema consulta el trazado y evalua estaciones cercanas.

Paso	Accion del usuario	Resultado esperado
4	Revisa el mapa y las tarjetas de parada.	Cada parada informa SoC, tiempo, energia, conector y costo estimado.
5	Si la bateria es baja, acepta la recomendacion de carga inicial.	Se agrega una electrolinera compatible cerca del origen.

Interpretar alertas

"Ruta no factible" significa que no existe una estacion compatible que pueda alcanzarse respetando la reserva. Aumenta el SoC, cambia de vehiculo o carga en una alternativa listada junto al origen. "Cobertura insuficiente" indica que el trazado no tiene continuidad de cargadores compatibles para completar el viaje.

Catalogo, cuenta y administracion

Desde Catalogo puedes filtrar vehiculos y electrolineras y guardar favoritos al iniciar sesion. En Perfil se consultan favoritos e historial. Las opciones administrativas permiten crear, editar o desactivar registros; solo deben usarse por cuentas autorizadas.

8. Ejecucion local y verificacion

Requisitos: Node.js 18 o superior y acceso a internet para el proveedor de ruteo OSRM. Desde la raiz del repositorio:

```
cd server npm install npm start cd .. cd client npm install npm run dev
```

La interfaz queda disponible en <http://127.0.0.1:5173> y la API en <http://127.0.0.1:3001>. Para una comprobacion rapida, abrir [/api/ready](#) y planificar Santiago -> Talca con Nissan Leaf e+ y 20% de bateria. El resultado debe presentar una carga inicial compatible.

Validacion realizada para esta version

Prueba	Resultado
npm test (server)	37 pruebas aprobadas.
npm run build (client)	Construccion Vite completada correctamente.
POST plan Santiago -> Talca, SoC 20%	Ruta exitosa; carga inicial en Copec Voltex Santiago Centro.
POST plan Santiago -> Talca, SoC 80%	Ruta exitosa; parada compatible en Curico.

9. Limitaciones y proximos pasos

El catalogo actual es una base local curada para el MVP. No consulta en tiempo real el estado operativo de cada cargador. La integracion de una fuente oficial o de operadores debe aplicarse mediante el flujo de sincronizacion administrativa y una validacion de calidad de datos.

Para produccion se recomienda: definir JWT_SECRET y variables en el proveedor de despliegue; usar una instancia propia o acordada de ruteo; conectar Supabase para persistencia real; y actualizar periodicamente estaciones, conectores, tarifas y disponibilidad.

Fin del documento.